

MANUAL PROFESIONAL · ÍNDICE MAESTRO

De cero absoluto a nivel experto



CLIENTES

id	nombre	email
1	Ana	ana@...
2	Luis	luis@...

```
SELECT nombre
FROM clientes
WHERE id = 1;

-- Tu 1ª consulta
```

BASES DE DATOS

MANUAL COMPLETO · SQL · POSTGRESQL · VPS · NUBE

Diseña · Crea · Consulta · Protege · Conecta · Despliega · Mantén · Escala

Índice general

Mapa de los 7 bloques, las 20 partes, el proyecto final y los anexos. El número entre paréntesis indica cuántos módulos tiene cada parte.

Cómo usar este manual

Bloque A — Fundamentos y diseño

Parte 1. Fundamentos absolutos	(10)
Parte 2. Cómo pensar una base de datos	(6)
Parte 3. Diseño de bases de datos	(8)
Parte 4. Normalización	(7)

Bloque B — SQL, PostgreSQL y herramientas

Parte 5. SQL desde cero	(19)
Parte 6. PostgreSQL	(14)
Parte 7. Herramientas para trabajar con PostgreSQL	(7)

Bloque C — Despliegue: VPS y nube

Parte 8. PostgreSQL en un VPS	(16)
Parte 9. Bases de datos en la nube	(6)

Bloque D — Plataformas y tecnologías

Parte 10. Supabase	(15)
Parte 11. Firebase	(12)
Parte 12. Airtable	(12)
Parte 13. Otras tecnologías	(3)

Bloque E — Elegir, conectar y autenticar

Parte 14. Cómo elegir una base de datos	(21)
Parte 15. Bases de datos y aplicaciones	(12)
Parte 16. Autenticación y usuarios	(15)

Bloque F — Seguridad, rendimiento y recuperación

Parte 17. Seguridad	(12)
Parte 18. Rendimiento	(13)
Parte 19. Backups y recuperación	(12)

Bloque G — Proyectos

Parte 20. Proyectos completos y progresivos	
Proyecto final: plataforma profesional	

Anexos

Comprobación de cobertura

Puntos a confirmar

Cómo usar este manual

Este documento es el índice maestro: el mapa completo del manual. Todavía no contiene los capítulos desarrollados. Sirve para revisar que no falta ninguna parte importante antes de escribirlos.

Cada concepto del manual se enseñará con la misma plantilla de 12 pasos, para que el alumno entienda lo que hace y no solo copie comandos.

1. ¿Qué es?
2. ¿Para qué sirve?
3. ¿Por qué lo necesito?
4. ¿Cómo funciona?
5. Ejemplo sencillo
6. Ejemplo real
7. Resultado visual (código junto a una imagen de la interfaz con lo construido)
8. Ejercicio
9. Solución
10. Error habitual
11. Buena práctica
12. Comprobación (cómo verificar que lo hecho funciona)

En la Parte 5 (SQL), cada concepto añade además: código completo, resultado esperado y explicación de ese resultado.

Regla de ejemplos visuales: cada ejemplo de cada punto lleva su código y, justo debajo, una imagen de la interfaz con el resultado de lo que se ha construido. Esto incluye tablas, relaciones, consultas, permisos, copias, pantallas de aplicación y flujos de n8n.

Reglas fijas de redacción: ningún término técnico sin explicar la primera vez que aparece; ritmo gradual; ningún conocimiento previo supuesto; explicaciones importantes sin recortar.

Niveles del recorrido

Nivel	Partes	Qué se consigue
1. Principiante	1 a 4	Entender los datos y diseñar un modelo sobre papel
2. Básico-intermedio	5 a 7	Crear y consultar bases de datos reales en PostgreSQL
3. Despliegue	8 y 9	Tener PostgreSQL propio o gestionado, funcionando
4. Plataformas	10 a 13	Usar Supabase, Firebase y Airtable, y conocer el resto
5. Arquitectura	14 a 16	Elegir tecnología, conectar aplicaciones y gestionar usuarios
6. Profesional	17 a 19	Proteger, optimizar y recuperar
7. Experto	20 y proyecto final	Construir sistemas completos de principio a fin

Política de ejemplos visuales

Cada ejemplo se presenta en tres piezas seguidas: el código completo, la imagen del resultado en la interfaz y una explicación de lo que se ve en la imagen. La imagen es la comprobación de que el alumno ha construido lo mismo.

Qué se construye	Imagen que acompaña
Tablas y datos	Vista de la tabla con sus filas, columnas y tipos, como en un cliente gráfico
Relaciones y claves	Diagrama entidad-relación con las claves y las líneas de relación
Resultado de una consulta SQL	Cuadrícula de resultados, antes y después de UPDATE y DELETE
Resultado de JOIN, agrupaciones y window functions	Tablas de origen y resultado lado a lado, con las filas coincidentes resaltadas
Plan de ejecución (EXPLAIN)	Plan en forma de árbol, antes y después de crear un índice
Usuarios, roles y permisos	Pantalla de roles y permisos, y el mensaje de error cuando se deniega un acceso
Terminal, SSH y VPS	Captura de la terminal con cada comando y su salida
Supabase, Firebase y Airtable	Pantalla de la plataforma: editor de tablas, reglas, vistas y registros enlazados
API, React y React Native	Respuesta JSON de la API y pantalla de la aplicación web y móvil con los datos
n8n	Flujo de nodos con su ejecución correcta
Backups y restauración	Salida de la copia y de la restauración, y comprobación de los datos recuperados
Proyectos	Secuencia de capturas por fase, con el estado de la base de datos y de la aplicación

La numeración será fija (Figura 5.3-a, 5.3-b...) y todas las figuras se recogerán en el Anexo H.

Bloque A — Fundamentos y diseño

Parte 1. Fundamentos absolutos

- 1.1 Dato frente a información: qué es cada uno y cómo un dato se convierte en información.
- 1.2 Por qué existen las bases de datos: el problema de las hojas de cálculo, los archivos sueltos y la información duplicada.
- 1.3 Qué es una base de datos y qué es un sistema gestor (SGBD); diferencia entre ambos.
- 1.4 Anatomía de una tabla con el ejemplo CLIENTES (id, nombre, email, teléfono): tabla, fila, columna, registro y campo.
- 1.5 Tipos de datos: texto, números enteros y decimales, fechas, verdadero/falso.
- 1.6 Valores, valor vacío (NULL) e identificadores.
- 1.7 Claves: qué es una clave, clave primaria y clave extranjera (primera presentación intuitiva).
- 1.8 Índices: la analogía del índice de un libro.
- 1.9 Consultas: qué es preguntar a una base de datos.
- 1.10 Conceptos del entorno explicados desde cero: servidor, cliente, API, VPS y base de datos en la nube (versión breve; se amplían en las Partes 8, 9 y 15).

Parte 2. Cómo pensar una base de datos

- 2.1 De la idea a los datos: las preguntas que hay que hacerse.
- 2.2 Cómo detectar entidades, atributos, relaciones y reglas en una descripción escrita.
- 2.3 Datos obligatorios y datos opcionales.
- 2.4 Caso guiado: «Quiero crear un CRM». Se descompone en usuarios, clientes, empresas, contactos, oportunidades, tareas, notas, llamadas, emails, estados y relaciones.
- 2.5 Otros casos de entrenamiento: agenda, tienda y sistema de reservas.
- 2.6 Plantilla reutilizable de análisis de requisitos de datos.

Parte 3. Diseño de bases de datos

- 3.1 Entidades y atributos.
- 3.2 Relaciones y cardinalidad: uno a uno, uno a muchos, muchos a muchos.
- 3.3 Tablas intermedias: por qué son necesarias (ejemplo cliente → pedidos → productos).
- 3.4 Claves primarias (natural, artificial, UUID) y claves extranjeras.
- 3.5 Restricciones: valores únicos, obligatorios, nulos y comprobaciones.
- 3.6 Diagramas entidad-relación: cómo leerlos y dibujarlos.
- 3.7 Del diagrama a las tablas.
- 3.8 Convenciones de nombres, fechas de creación y modificación, y borrado lógico.

Parte 4. Normalización

- 4.1 El problema: redundancia y anomalías de inserción, actualización y borrado.
- 4.2 Primera forma normal (1FN), con ejemplo antes y después.
- 4.3 Segunda forma normal (2FN), con ejemplo antes y después.
- 4.4 Tercera forma normal (3FN), con ejemplo antes y después.
- 4.5 Cómo reconocer el problema en una tabla real.
- 4.6 Cuándo normalizar y cuándo puede tener sentido desnormalizar.
- 4.7 Errores habituales de diseño: columnas repetidas, listas dentro de una celda, mezclar entidades.

Bloque B — SQL, PostgreSQL y herramientas

Parte 5. SQL desde cero

Cada punto lleva: explicación sencilla, ejemplo, código completo, resultado esperado, explicación del resultado, ejercicio, solución y errores frecuentes.

- 5.0** Qué es SQL, cómo se lee una sentencia y el orden lógico de ejecución de una consulta. Base de datos de práctica que se usa en todo el bloque.
- 5.1** Crear: CREATE DATABASE, CREATE TABLE, tipos de datos, claves, secuencias e identidades.
- 5.2** Insertar: INSERT (una fila y varias).
- 5.3** Leer: SELECT, alias, DISTINCT, ORDER BY, LIMIT (y OFFSET).
- 5.4** Filtrar: WHERE, AND, OR, NOT, IN, BETWEEN, LIKE, IS NULL.
- 5.5** Modificar y borrar: UPDATE y DELETE, con la regla de seguridad «probar primero con SELECT».
- 5.6** Resumir: COUNT, SUM, AVG, MIN, MAX, GROUP BY, HAVING.
- 5.7** Combinar tablas: JOIN, INNER JOIN, LEFT JOIN, RIGHT JOIN, FULL JOIN, CROSS JOIN.
- 5.8** Combinar resultados: UNION (y UNION ALL), INTERSECT y EXCEPT.
- 5.9** Subconsultas y CTE (WITH).
- 5.10** Condiciones y nulos: CASE, COALESCE, NULLIF.
- 5.11** Funciones de texto, numéricas y de fecha.
- 5.12** Window functions: OVER, PARTITION BY, ROW_NUMBER, RANK, LAG, LEAD, totales acumulados.
- 5.13** CTE recursivas (jerarquías, por ejemplo categorías y subcategorías).
- 5.14** Estructura avanzada: vistas, índices y restricciones (PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT).
- 5.15** Transacciones: BEGIN, COMMIT, ROLLBACK y por qué existen (propiedades ACID explicadas).
- 5.16** Programar dentro de la base: funciones, procedimientos y triggers.
- 5.17** Errores frecuentes de SQL y cómo leer un mensaje de error.
- 5.18** Práctica integrada: ejercicios de repaso y mini examen.

Parte 6. PostgreSQL

- 6.1** Qué es PostgreSQL, historia breve y por qué se utiliza.
- 6.2** Instalación en el ordenador (Windows, macOS, Linux y Docker) y primera comprobación.
- 6.3** Arquitectura básica: servidor, clusters, bases de datos, schemas y tablas.
- 6.4** Usuarios, roles y permisos (GRANT y REVOKE).
- 6.5** Schemas.
- 6.6** Tipos de datos propios de PostgreSQL: texto, numéricos, booleanos, fechas y timestamps con zona horaria, UUID, arrays, JSON y JSONB, enumerados.
- 6.7** Relaciones y constraints en PostgreSQL, incluido ON DELETE y ON UPDATE.

- 6.8 Índices: B-tree, GIN, parciales y compuestos.
- 6.9 Transacciones y aislamiento a nivel práctico.
- 6.10 Extensiones (por ejemplo uuid-ossdp o pgcrypto, y otras relevantes).
- 6.11 Funciones, triggers y vistas en PostgreSQL; materialized views.
- 6.12 Backups y restauración: pg_dump y pg_restore (introducción; el detalle va en la Parte 19).
- 6.13 Rendimiento: EXPLAIN y EXPLAIN ANALYZE (introducción; el detalle va en la Parte 18).
- 6.14 Ejercicios prácticos de PostgreSQL.

Parte 7. Herramientas para trabajar con PostgreSQL

- 7.1 Visión general: cuándo usar cada herramienta y para qué sirve.
- 7.2 psql, el cliente de terminal: conexión, comandos \d, \l y \dt, y ejecución de scripts.
- 7.3 pgAdmin.
- 7.4 DBeaver.
- 7.5 Otras herramientas gráficas relevantes y clientes desde VS Code.
- 7.6 Tabla comparativa: tarea, herramienta recomendada y motivo.
- 7.7 Buenas prácticas de trabajo: scripts versionados, migraciones y entornos de desarrollo y producción.

Bloque C — Despliegue: VPS y nube

Parte 8. PostgreSQL en un VPS

- 8.1 Conceptos previos desde cero: qué es un servidor, un VPS, Linux, la terminal y SSH.
- 8.2 Elegir y contratar un VPS: qué mirar (CPU, memoria, disco, ubicación), sin atar el manual a un proveedor concreto.
- 8.3 Primer acceso por SSH: claves frente a contraseñas, usuario no administrador y sudo.
- 8.4 Preparar el servidor: actualizaciones, zona horaria y firewall.
- 8.5 Instalar PostgreSQL en Linux.
- 8.6 Configuración: los archivos postgresql.conf y pg_hba.conf, qué hace cada uno.
- 8.7 Usuarios, bases de datos y permisos para una aplicación.
- 8.8 Acceso remoto: cuándo abrirlo y cuándo no, túnel SSH, lista de IP permitidas y TLS.
- 8.9 Seguridad del servidor (resumen; el detalle va en la Parte 17).
- 8.10 Backups y restauración en el VPS (resumen; el detalle va en la Parte 19).
- 8.11 Actualizaciones de PostgreSQL: versiones menores y mayores.
- 8.12 Monitorización básica: espacio en disco, conexiones, logs y consultas lentas.
- 8.13 Conexión desde una aplicación: la cadena de conexión explicada pieza a pieza.
- 8.14 Conexión desde Node.js, desde n8n y desde un panel administrativo.
- 8.15 Ejemplo completo en capas: VPS → PostgreSQL → API Node.js → React → React Native, con qué ocurre en cada capa y dónde falla cada una.
- 8.16 Lista de comprobación de un VPS listo para producción.

Parte 9. Bases de datos en la nube

- 9.1 Qué es una base de datos gestionada y qué se delega al proveedor.
- 9.2 Cuatro modelos comparados: servidor propio, VPS, base de datos gestionada y backend como servicio (BaaS).
- 9.3 Tabla comparativa: ventajas, inconvenientes, costes, mantenimiento y casos de uso.
- 9.4 Conceptos clave: regiones, latencia, límites de conexiones, pooling, escalado, copias del proveedor y dependencia del proveedor (vendor lock-in).
- 9.5 Cumplimiento y ubicación de los datos: RGPD y región de la UE, explicado de forma práctica.
- 9.6 Cómo migrar de un modelo a otro.

Bloque D — Plataformas y tecnologías

Parte 10. Supabase

- 10.1** Qué es Supabase, cómo funciona y qué piezas incluye.
- 10.2** PostgreSQL dentro de Supabase: qué es igual que en la Parte 6 y qué añade la plataforma.
- 10.3** Crear un proyecto y recorrer el panel; editor de tablas y editor SQL.
- 10.4** Tablas y relaciones desde el panel y desde SQL; migraciones.
- 10.5** Autenticación y usuarios: el esquema auth y cómo enlazarlo con tus tablas.
- 10.6** Row Level Security y políticas: qué son, cómo se escriben y cómo se prueban.
- 10.7** Storage: buckets, archivos y políticas de acceso.
- 10.8** APIs generadas automáticamente y librería cliente.
- 10.9** Realtime.
- 10.10** Edge Functions.
- 10.11** Backups y recuperación en Supabase.
- 10.12** Variables y claves: cuál es pública, cuál es secreta y dónde se guarda cada una.
- 10.13** Conexión con React, con React Native, con Node.js y con n8n.
- 10.14** Proyectos prácticos con Supabase.
- 10.15** Cuándo elegir Supabase y cuándo no.

Parte 11. Firebase

- 11.1** Qué es Firebase y qué servicios incluye.
- 11.2** Firebase Authentication.
- 11.3** Firestore: documentos, colecciones y subcolecciones.
- 11.4** Estructura de datos en Firestore: cómo modelar sin tablas ni JOIN.
- 11.5** Consultas e índices en Firestore y sus límites.
- 11.6** Realtime Database y diferencias con Firestore.
- 11.7** Storage.
- 11.8** Cloud Functions.
- 11.9** Reglas de seguridad.
- 11.10** Firestore frente a PostgreSQL: tabla comparativa de modelo, consultas, relaciones, costes y escalado.
- 11.11** Cuándo usar Firebase y cuándo no.
- 11.12** Ejemplos prácticos con Firebase.

Parte 12. Airtable

- 12.1** Qué es Airtable y por qué no es «una hoja de Excel»: es una base de datos relacional con interfaz visual, y se enseña como tal.
- 12.2** Bases, tablas, campos y registros.

- 12.3** Tipos de campo.
- 12.4** Vistas: cuadrícula, kanban, calendario, galería y filtros.
- 12.5** Relaciones: linked records, lookup y rollup.
- 12.6** Fórmulas.
- 12.7** Automatizaciones.
- 12.8** Interfaces.
- 12.9** API de Airtable y webhooks cuando corresponda.
- 12.10** Conexión con n8n y con otras aplicaciones.
- 12.11** Límites (registros, llamadas, coste) y cuándo migrar a PostgreSQL.
- 12.12** Ejemplos prácticos con Airtable.

Parte 13. Otras tecnologías

Criterio de selección: solo tecnologías relevantes hoy para un desarrollador en España. Antes de escribir esta parte se investigará y verificará con fuentes el estado actual de cada una. No habrá rankings de «mejor o peor»: el objetivo es aprender a elegir según las necesidades del proyecto.

Cada tecnología se describe con la misma ficha: qué es, tipo de base de datos, características, ventajas, inconvenientes, curva de aprendizaje, casos de uso, ejemplos, integración con aplicaciones, integración con n8n, escalabilidad, costes generales, cuándo elegirla y cuándo no.

Tipo	Tecnologías previstas
Relacionales	PostgreSQL (repaso), MySQL, MariaDB, SQLite, SQL Server; Oracle y otras solo si la investigación confirma su relevancia
NoSQL	MongoDB, Redis, Firestore; otras (por ejemplo búsqueda o series temporales) solo si se justifica
Backend como servicio y plataformas	Supabase y Firebase (repaso) y otras relevantes por verificar
Low-code y no-code	Airtable (repaso) y otras con utilidad real por verificar

- 13.1** Ficha de cada tecnología, agrupada por tipo.
- 13.2** Tabla de integración con n8n: nodo nativo, API o conexión directa, según cada tecnología (se verificará antes de escribirla).
- 13.3** Conceptos transversales: relacional frente a NoSQL, ACID frente a consistencia eventual, y por qué muchas arquitecturas combinan varias bases de datos.

Bloque E — Elegir, conectar y autenticar

Parte 14. Cómo elegir una base de datos

- 14.1** El método de decisión en pasos: describir el proyecto, puntuar los factores, descartar opciones, validar con un prototipo.
- 14.2** Factores a analizar: volumen, relaciones, usuarios, seguridad, rendimiento, escalabilidad, presupuesto, mantenimiento, velocidad de desarrollo, tiempo real, integración y conocimientos técnicos.
- 14.3** Sistema de decisión «Quiero crear...». Para cada escenario se explican los factores que pesan más y las opciones que encajan:
 - 14.3.1** Una aplicación sencilla.
 - 14.3.2** Una aplicación móvil.
 - 14.3.3** Un CRM.
 - 14.3.4** Un SaaS.
 - 14.3.5** Un sistema con muchos usuarios.
 - 14.3.6** Una automatización.
 - 14.3.7** Un proyecto interno de empresa.
 - 14.3.8** Un prototipo rápido.
 - 14.3.9** Un proyecto profesional.
 - 14.3.10** Una aplicación con datos relacionales complejos.
 - 14.3.11** Un sistema que necesita tiempo real.
 - 14.3.12** Una aplicación que necesita autenticación.
 - 14.3.13** Una aplicación con archivos.
 - 14.3.14** Un proyecto en VPS.
 - 14.3.15** Un proyecto con presupuesto reducido.
- 14.4** Matriz de necesidades frente a tecnologías (sin rankings).
- 14.5** Casos resueltos paso a paso, incluido «CRM SaaS para varias empresas».
- 14.6** Errores de elección frecuentes y cómo migrar si te equivocas.

Parte 15. Bases de datos y aplicaciones

- 15.1** Por qué normalmente no se conecta una aplicación pública directamente a una base de datos privada.
- 15.2** La arquitectura estándar: APLICACIÓN → API / BACKEND → BASE DE DATOS, y qué hace cada capa.
- 15.3** Qué es una API REST: rutas, métodos, códigos de respuesta y JSON.
- 15.4** Node.js y PostgreSQL: conexión, pool de conexiones y consultas parametrizadas.
- 15.5** React: cómo consume la API (peticiones, estados de carga y errores).
- 15.6** React Native: particularidades móviles (red, almacenamiento seguro de tokens, modo sin conexión).
- 15.7** APIs externas: consumirlas y guardar sus datos.

- 15.8** n8n: nodos de base de datos, webhooks y conexión a PostgreSQL.
- 15.9** Automatizaciones: patrones habituales (sincronizar, notificar, importar).
- 15.10** Paneles administrativos.
- 15.11** Cuándo existen arquitecturas diferentes: BaaS con Row Level Security, funciones serverless, acceso directo controlado y GraphQL.
- 15.12** Variables de entorno, CORS y errores comunes de conexión.

Parte 16. Autenticación y usuarios

- 16.1** Qué es un usuario y cómo se modela en tablas.
- 16.2** Registro.
- 16.3** Login.
- 16.4** Contraseñas y hash: por qué nunca se guardan en claro, sal (salt) y algoritmos adecuados.
- 16.5** Sesiones.
- 16.6** Tokens.
- 16.7** JWT: qué contiene, cómo se firma y qué no debe contener.
- 16.8** Refresh tokens.
- 16.9** Roles.
- 16.10** Permisos y autorización: diferencia entre autenticación y autorización.
- 16.11** Multiusuario.
- 16.12** Multiempresa: el identificador de empresa (tenant) en cada tabla y su aislamiento.
- 16.13** Recuperación de contraseña.
- 16.14** Complementos habituales: inicio de sesión con terceros (OAuth) y verificación en dos pasos.
- 16.15** Modelo completo de tablas de usuarios, roles y permisos.

Bloque F — Seguridad, rendimiento y recuperación

Parte 17. Seguridad

- 17.1** Modelo de amenazas explicado desde cero: qué proteger y de quién.
- 17.2** SQL injection: cómo ocurre, ejemplo vulnerable y consultas parametrizadas.
- 17.3** Permisos y mínimos privilegios.
- 17.4** Credenciales, variables de entorno y secretos.
- 17.5** Exposición de bases de datos y acceso público.
- 17.6** Row Level Security.
- 17.7** Cifrado en reposo y en tránsito.
- 17.8** Conexiones seguras (TLS).
- 17.9** Seguridad de los backups.
- 17.10** Auditoría y registros (logs).
- 17.11** Protección de datos personales (RGPD) a nivel práctico.
- 17.12** Lista de comprobación de seguridad.

Parte 18. Rendimiento

- 18.1** Cómo se ejecuta una consulta y por qué algunas son lentas.
- 18.2** Índices: cuándo ayudan y cuándo estorban.
- 18.3** Detectar consultas lentas.
- 18.4** EXPLAIN y EXPLAIN ANALYZE: leer un plan de ejecución.
- 18.5** Joins eficientes.
- 18.6** Paginación: OFFSET frente a paginación por clave.
- 18.7** Caché.
- 18.8** El problema N+1.
- 18.9** Consultas innecesarias.
- 18.10** Grandes volúmenes: particionado y archivado.
- 18.11** Pool de conexiones.
- 18.12** Casos de consultas mal planteadas, con análisis y mejora.
- 18.13** Lista de comprobación de rendimiento.

Parte 19. Backups y recuperación

- 19.1** Qué es un backup y qué se pierde sin él.
- 19.2** Tipos: completo, incremental y diferencial.
- 19.3** Copia lógica frente a copia física: pg_dump, pg_dumpall y pg_basebackup.

- 19.4** Recuperación a un punto en el tiempo (WAL y PITR), explicado desde cero.
- 19.5** Estrategia de copias: la regla 3-2-1, frecuencia y retención.
- 19.6** Backups automáticos: tareas programadas y verificación.
- 19.7** Restauración paso a paso.
- 19.8** Recuperación ante errores: borrado accidental, disco lleno y servidor perdido.
- 19.9** Pruebas de restauración.
- 19.10** Ejemplo real: PostgreSQL en un VPS con copia diaria cifrada y enviada a otro lugar.
- 19.11** Backups en plataformas gestionadas.
- 19.12** Plan de recuperación por escrito.

Bloque G — Proyectos

Parte 20. Proyectos completos y progresivos

Cada proyecto incluye: objetivo, modelo de datos, SQL, conceptos que practica y comprobación final.

Proyecto	Descripción	Qué se practica
1	Agenda de contactos	Tablas, INSERT, SELECT, filtros
2	Gestor de tareas	Estados, fechas, UPDATE y DELETE, ordenación
3	Sistema de usuarios	Claves únicas, hash de contraseñas, roles básicos
4	Sistema de clientes	Relaciones uno a muchos, JOIN, agregaciones
5	Mini CRM	Varias entidades, oportunidades, notas, vistas
6	Sistema de reservas	Restricciones, fechas, solapamientos, transacciones
7	Aplicación móvil conectada a una base de datos	React Native, API y tokens
8	Aplicación React + API + PostgreSQL	Arquitectura en capas completa
9	CRM con usuarios, empresas, clientes, tareas y permisos	Multiempresa, roles y autorización
10	SaaS completo	Planes, suscripciones, aislamiento por empresa, despliegue

Proyecto final: plataforma profesional

Objetivo: construir progresivamente una plataforma completa con estas piezas.

Pieza	Tecnología
Frontend	React
Aplicación móvil	React Native
Backend	Node.js
Base de datos	PostgreSQL
Servidor	VPS
Automatizaciones	n8n

También incluye: autenticación, usuarios, roles, permisos, API, CRUD, relaciones, archivos, backups, logs, seguridad y despliegue.

- F.0** Requisitos y análisis de datos (Partes 2 y 3).
- F.1** Modelo de datos y script SQL inicial (Partes 3 a 5).
- F.2** PostgreSQL en el VPS (Parte 8).
- F.3** API Node.js con CRUD y relaciones (Parte 15).
- F.4** Usuarios, roles y permisos (Parte 16).
- F.5** Frontend React (Parte 15).
- F.6** Aplicación React Native (Parte 15).
- F.7** Archivos y almacenamiento.
- F.8** Automatizaciones con n8n.
- F.9** Seguridad y endurecimiento (Parte 17).
- F.10** Rendimiento y revisión de consultas (Parte 18).
- F.11** Backups, restauración y pruebas (Parte 19).
- F.12** Registros (logs) y monitorización.
- F.13** Despliegue y puesta en producción.
- F.14** Mantenimiento, escalado y siguientes pasos.

Anexos

- A.** Glosario completo de términos, con explicación sencilla.
- B.** Chuleta de SQL y de psql.
- C.** Plantillas: análisis de requisitos de datos, ficha de tecnología, lista de comprobación del VPS y plan de recuperación.
- D.** Soluciones de todos los ejercicios.
- E.** Scripts de las bases de datos de práctica.
- F.** Mapa de dependencias entre partes (qué leer antes de cada una).
- G.** Documentación oficial recomendada de cada tecnología.
- H.** Índice de figuras: todas las imágenes de resultados, numeradas y enlazadas con su ejemplo.

Comprobación de cobertura

La tabla cruza cada punto de la petición original con la parte que lo cubre.

Petición	Dónde se cubre
Qué es una base de datos y por qué existen	Parte 1
Cómo funcionan	Partes 1, 5 y 6
Cómo se diseñan y se relacionan los datos	Partes 2, 3 y 4
Cómo elegir una tecnología	Partes 13 y 14
Cómo crear, consultar y modificar datos	Partes 5 y 6
Cómo protegerlos	Partes 16 y 17
Cómo conectarlos con aplicaciones	Parte 15 y proyectos
Cómo desplegarlos	Partes 8, 9 y 10
Cómo mantenerlos	Partes 8, 18 y 19
Cómo escalarlos	Partes 9, 18 y proyecto final F.14
PostgreSQL en profundidad y herramientas	Partes 6 y 7
PostgreSQL propio en un VPS	Parte 8
Supabase, Firebase y Airtable	Partes 10, 11 y 12
Otras tecnologías relevantes en España	Parte 13
Conexión con React, React Native, Node.js y n8n	Partes 8, 10, 15 y proyectos 7 a 10
Usuarios, permisos y multiempresa	Parte 16 y proyecto 9
Backups y recuperación	Parte 19
Proyectos progresivos y proyecto final	Parte 20
Plantilla didáctica de 12 pasos	Cómo usar este manual (aplicada en todas las partes)
Ejemplos de código con imagen del resultado en la interfaz	Política de ejemplos visuales, todas las partes y Anexo H

Puntos a confirmar

1. Portada: pediste una portada «como el resto», pero no he encontrado otros manuales tuyos en este repositorio, así que he diseñado una nueva. Si tienes un modelo, indícalo y la adapto.
2. Parte 13: se escribirá con investigación y fuentes verificadas; hasta entonces la lista de tecnologías es provisional.
3. Versiones: las instrucciones de instalación se escribirán con las versiones vigentes en el momento de redactar cada capítulo.
4. Proveedor de VPS: el manual será neutral; dime si quieres ejemplos con uno concreto.
5. Imágenes de resultados: donde la herramienta pueda ejecutarse en el entorno de trabajo (por ejemplo PostgreSQL con un cliente gráfico), las imágenes serán capturas reales de lo ejecutado. Donde no pueda (por ejemplo plataformas de terceros con cuenta propia), serán maquetas fieles, rotuladas como tales; no se presentará una maqueta como captura real.
6. Formato final: ahora hay PDF y DOCX del índice; los capítulos podrían entregarse igual, en un solo archivo o uno por parte.